CSCI 12532
Computer Architecture and Design
Lecture 03

Ms. Meesha Mervyn
Dept. of Computer Systems Engineering
Faculty of Computing and Technology
University of Kelaniya
meesham@kln.ac.lk

Contents

1. PROCESSOR ORGANIZATION

Processor Organization Refers to the structure and design of the Central Processing Unit (CPU), which is the core component of a computer system. Determines how efficiently and effectively a computer can execute instructions and process data and Plays a critical role in the overall performance, speed, and functionality of a computer.

Why is processor organization is important

Directly influences the speed and accuracy of instruction execution. A well-designed organization ensures efficient fetching, decoding, and execution of instructions.

Optimizes the design and coordination of CPU components (e.g., control unit, ALU, registers, cache memory). Enhances the ability to handle complex calculations and tasks quickly.

Determines how the CPU accesses and utilizes different types of memory (cache, main memory, secondary storage). Ensures efficient allocation and management of system resources.

Implements techniques such as

- Clock Gating: Reduces power consumption by disabling clock signals to inactive components.
- Power Gating: Shuts off power to unused parts of the processor.
- Dynamic Voltage Scaling: Adjusts voltage and frequency based on workload to save energy.

Requirements expected of a Processor

- Fetch instruction: reads an instruction from memory;
- Interpret instruction: determines what action to perform;
- Fetch data: if necessary read data from memory or an I/O module.
- Process data: If necessary perform arithmetic / logical operation on data.
- Write data: If necessary write data to memory or an I/O module.

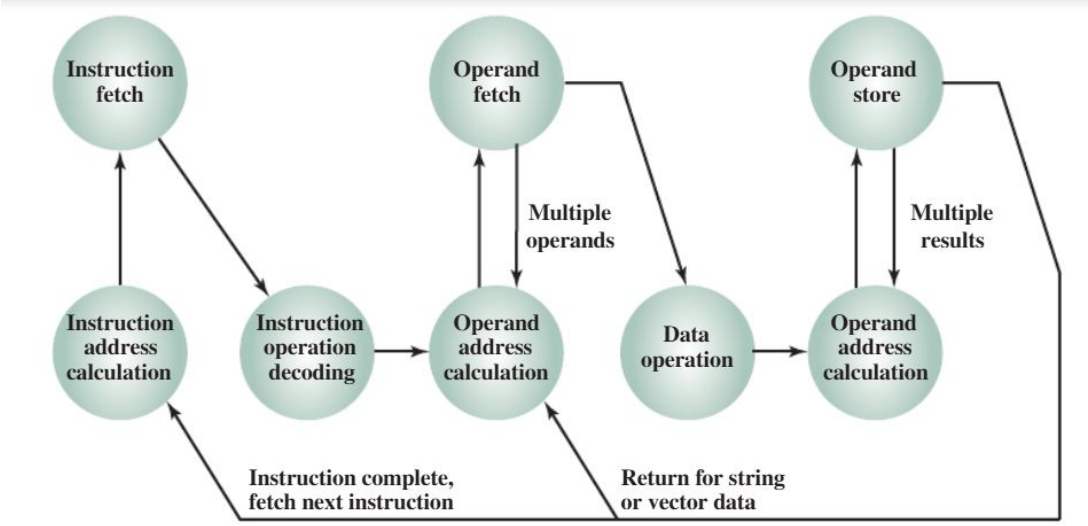
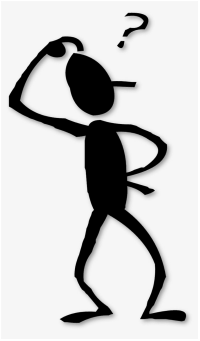


Figure 12.1 Instruction Cycle State Diagram

So what components does a CPU need to perform these requirements?



Internal Structure of the CPU

Major components of the processor:

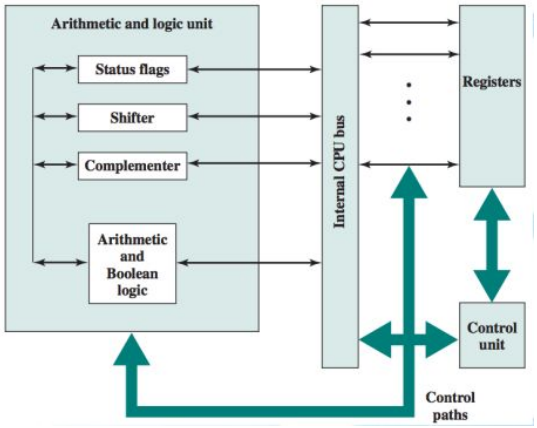
• Arithmetic and Logic Unit (ALU): Performs computation or processing of data

• Control Unit: Moves data and instructions in and out of the processor; Also controls the operation of the ALU;

• Registers: Used as internal memory;

• System Bus: Acting as a pathway between processor, memory and I/O module(s);

A detailed overview of the CPU



The role of the internal components

Arithmetic Logic Unit (ALU): The ALU is responsible for performing arithmetic operations (such as addition and subtraction) and logical operations (such as AND, OR, and NOT). It is the primary unit for executing data manipulation instructions within the CPU.

Control Unit: This unit directs the operations of the CPU, managing the flow of data between internal components and coordinating tasks such as fetching, decoding, and executing instructions.

Registers: Registers are small, high-speed storage locations within the CPU used to hold data, instructions, or addresses during processing temporarily. Common types include:

- Accumulator (AC): Used to store intermediate results of arithmetic and logic operations.
- Program Counter (PC): Holds the address of the next instruction to be fetched from memory.
- Instruction Register (IR): Stores the current instruction being executed.
- Address Register (AR): Contains memory addresses used for data transfer.

Cache: Cache memory is a small, high-speed memory unit located near the CPU core. It stores frequently accessed data and instructions, reducing the time needed to fetch them from main memory.

Clock: The clock generates timing signals that synchronize the operations of all CPU components, ensuring data is processed in a coordinated manner.

Data Bus: The data bus is a set of parallel wires or pathways that transfer data between the CPU, memory, and other hardware components.

Multiplexer (MUX): A multiplexer selects one of several input signals and forwards the chosen input into a single line, helping manage data flow within the CPU.

Flip-Flops: Flip-flops are basic storage elements used to store individual bits of data within registers and other control circuits.

Types of CPU Organization in Computer Architecture

1. Single Accumulator Organization

This is one of the simplest forms of CPU design. In this organization, there is only one accumulator register that holds intermediate data for arithmetic or logical operations. The CPU fetches data from memory into the accumulator, performs the operation, and stores the result back into the accumulator or memory.

2. General Register Organization

The system described is a CPU bus organization that uses seven registers connected to two multiplexers (MUX), forming two buses, A and B. These buses connect to an Arithmetic Logic Unit (ALU), which performs various arithmetic or logic operations based on control signals. The result is then routed to the output bus, which feeds back into the registers, with one selected register receiving the result.

Registers and Buses

- Each register is connected to two multiplexers (MUX).
- The output of these registers is sent to buses A and B through the multiplexers.
- The MUX selection lines determine which register data is placed onto each bus.

ALU (Arithmetic Logic Unit)

- Buses A and B feed the ALU, which performs operations like addition or subtraction depending on the control signal.
- The ALU's output is then sent to the output bus.

Register Load

- A decoder controls which register will receive the ALU's result from the output bus.
- The destination register is selected via the decoder, which activates the load input of the chosen register.

Control Unit

The control unit generates four key control signals:

- MUX A selector (SELA): Chooses the source register for bus A.
- MUX B selector (SELB): Chooses the source register for bus B.
- ALU operation selector (OPR): Defines the ALU operation (e.g., addition).
- Decoder destination selector (SELD): Chooses which register will load the result.

Example of Operation

For the operation R1 <- R2 + R3, the control signals would:

1. Set SELA to select R2 for bus A.
2. Set SELB to select R3 for bus B.
3. Set OPR to perform addition in the ALU.
4. Set SELD to select R1 as the destination register.

These control signals direct data flow from registers to the ALU and then into the selected register during the clock cycle.

3. Stack Organization

A stack is a fundamental data structure in computer architecture, used to manage memory in a Last-In-First-Out (LIFO) manner. It is commonly employed for function calls and local variables, simplifying memory management and improving execution efficiency.

Key Components

- Stack Pointer (SP): A register that holds the address of the top element on the stack.
- Push Operation: Inserts an element onto the stack, incrementing the SP register.
- Pop Operation: Deletes the top element from the stack, decrementing the SP register.

2. REGISTER ORGANIZATION

Registers in the processor perform two roles

- User-visible registers:
  - Used as internal memory by the assembly language programmer;
- Control and status registers:
  - Used to control the operation of the processor;
  - Used to check the status of the processor / ALU

User-visible Registers

May be referenced by the programmer, categorized into:

- General purpose
- Data
- Address
- Condition codes

General-purpose registers

Can be assigned to a variety of functions by the programmer:

- Memory reference & backup;
- Register reference & backup;
- Data reference & backup;

Data registers

May be used only to hold data and cannot hold addresses:

- Must be able to hold values of most data types;
- Some machines allow two contiguous registers to be used:
- For holding double-length values.

Address Registers

Used to hold addresses, e.g.:

- Stack Pointer
- Program Counter
- Index Registers

Must be at least long enough to hold the largest address

Condition Codes

Hold condition codes (a.k.a. flags):

- Flags are bits set by processor as the result of operations
- E.g.: an arithmetic operation may produce:
    - a positive result;
    - a negative result;
    - a zero result;
    - an overflow result.

Condition code bits are collected into one or more control registers:
In some machines:
- Interruption results in all user-visible registers being saved; These are then restored on return;
- Allows each subroutine to use the user-visible registers independently;

On other machines:
- responsibility of the programmer to Save the contents of user- visible registers prior to a subroutine call;

Control and Status Registers

Employed to control the operation of the processor:

- Mostly not visible to the user;
- Program counter (PC): Contains instruction address to be fetched;
- Instruction Register (IR): Contains last instruction fetched;
- Memory address register (MAR): Contains memory location address;
- Memory buffer register (MBR): Contains:
    - Word of data to be written to memory;
    - Word of data read from memory.

In general terms:

- Processor updates PC after each instruction fetch;
- A branch or skip instruction will also modify the contents of the PC;
- The fetched instruction is loaded into an IR
- Data are exchanged with memory using the MAR and MBR, e.g.:
    - MAR connects directly to the address bus
    - MBR connects directly to the data bus

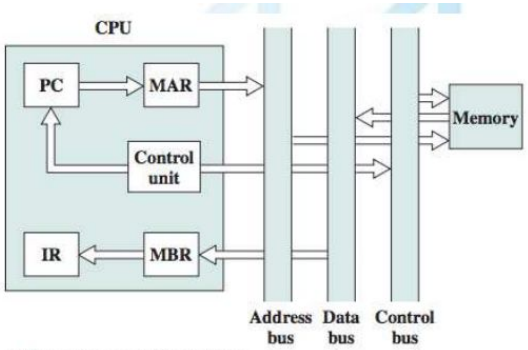
The four registers just mentioned are used for:

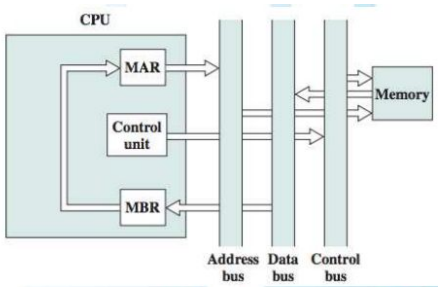
- Data movement between processor and memory;
- Within the processor, data must be presented to the ALU for processing:
  - ALU may have direct access to the MBR and user-visible registers;
  - Alternatively:
    - There may be additional buffering registers within ALU;
    - These registers serve as input and output registers for the ALU;
    - These registers exchange data with the MBR and user-visible registers.

Many processors include a program status word (PSW) register:

- Contains condition codes plus other status information
- Common fields or flags include the following:
  - Sign: Sign bit of the result of the last arithmetic operation;
  - Zero: when the result is 0;
  - Carry: Set if an operation resulted in a carry/borrow bit;
  - Equal: Set if a logical compare result is equality.
  - Overflow: Used to indicate arithmetic overflow.
  - Interrupt Enable/Disable: Used to enable or disable interrupts.

Fetch and decode Cycle





The flow of data during the instruction fetch cycle:

1 PC contains the address of the next instruction to be fetched;
2 Address is moved to the MAR and placed on the address bus;
3 Control unit requests a memory read;
4 Result is:
- placed on the data bus;
- copied into the MBR;
- then moved to the IR.

5 Meanwhile, the PC is incremented by 1;

Once the fetch cycle is over, control unit examines IR:

1 to determine if it contains an operand specifier using indirect addressing;
2 If so, an indirect cycle is performed:
The indirect addressing cycle:
- Bits of the MBR containing the address are transferred to the MAR;
- Control unit then requests a memory read:
- to get the desired address of the operand into the MBR.

The execute cycle takes many forms:

• Depending on the operation to be performed...

• May involve:

  • Transferring data among registers

  • Read or write from memory or I/O

  • And/or the invocation of the ALU.

Register Organization Techniques

Aim to improve performance and efficiency by optimizing the usage of registers within a processor.
Focus on reducing data dependencies, minimizing memory access, and enhancing instruction-level parallelism.

1. Register Renaming

- Reduce data dependencies and improve instruction-level parallelism.
- Assigns temporary or virtual registers to hold intermediate values during program execution.
- Dynamically maps original registers to temporary registers to resolve dependencies.
- Benefits;
  - Allows multiple instructions with dependencies on the same register to execute simultaneously.
  - Enhances performance by enabling parallel execution of instructions.
- Commonly used in out-of-order execution processors to improve throughput.

2. Register Allocation

- Optimize the assignment of registers to variables within a program.
- Compiler algorithms analyze the program's variable usage patterns.
- Assign registers to frequently accessed variables to minimize memory accesses.
- Benefits;
  - Maximizes the use of registers for storing critical variables.
  - Reduces memory latency and improves overall performance.
- Used in compiler optimization to ensure efficient use of CPU registers.

3. Register Windowing

- Increase the number of available registers using a sliding window approach.
- Utilizes a subset of registers (a "window") that is accessible at a given time.
- Slides the window to make different sets of registers visible as needed.
- Benefits;
  - Allows more variables to be stored in registers, reducing memory access overhead.
  - Efficiently manages a large number of physical registers with limited visibility.
- Commonly used in RISC processors to handle function calls and context switching.

4. Register Stack:

- Efficiently manage temporary values and function calls using a stack-based organization.
- Registers are pushed onto a stack when a function is called and popped when the function returns.
- Saves and restores registers during context switching.
- Benefits;
  - Increases the effective number of available registers.
  - Reduces memory traffic by minimizing data movement between registers and memory.
  - Facilitates efficient function calls and recursion.
- Used in processors to handle nested function calls and interrupt handling.

Advantages of Register Organization Techniques

- Improved Performance:
  Register organization techniques, such as register renaming and allocation, reduce dependencies and increase instruction-level parallelism, leading to faster execution and improved performance.

- Reduced Memory Latency:
  By maximizing the use of registers for frequently accessed variables, these techniques minimize the need for frequent memory accesses, reducing memory latency and improving overall system performance.

- Efficient Resource Utilization:
  Techniques like register windowing and register stack optimize the utilization of available registers, increasing the effective number of registers and improving the efficiency of register usage.

Thank You

10.04.2026